

一套自研的三层图工程

不是通用图引擎框架，而是由确定性策略、DAG 算法和持久化账本共同组成。



共同运行底座

状态真源：后端投影 legal_actions / next_action

可靠性：command_id 幂等 + expected_state_version 乐观锁

一致性：prepare / materialize 两阶段命令 + 单事务落库

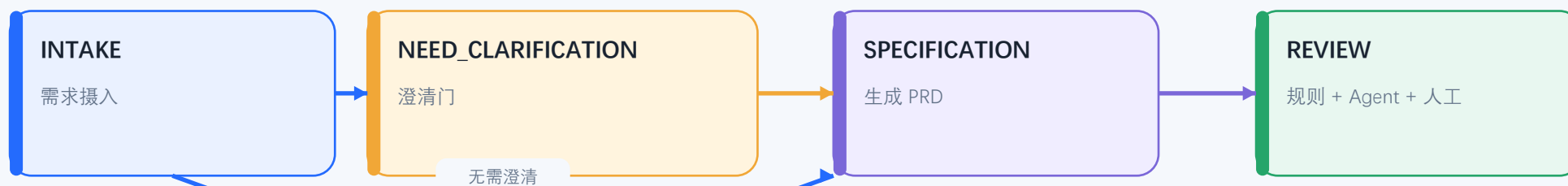
审计性：AgentCall、CommandAttempt、ProcessedCommand、AuditEvent、WorkItemRun

核心规律

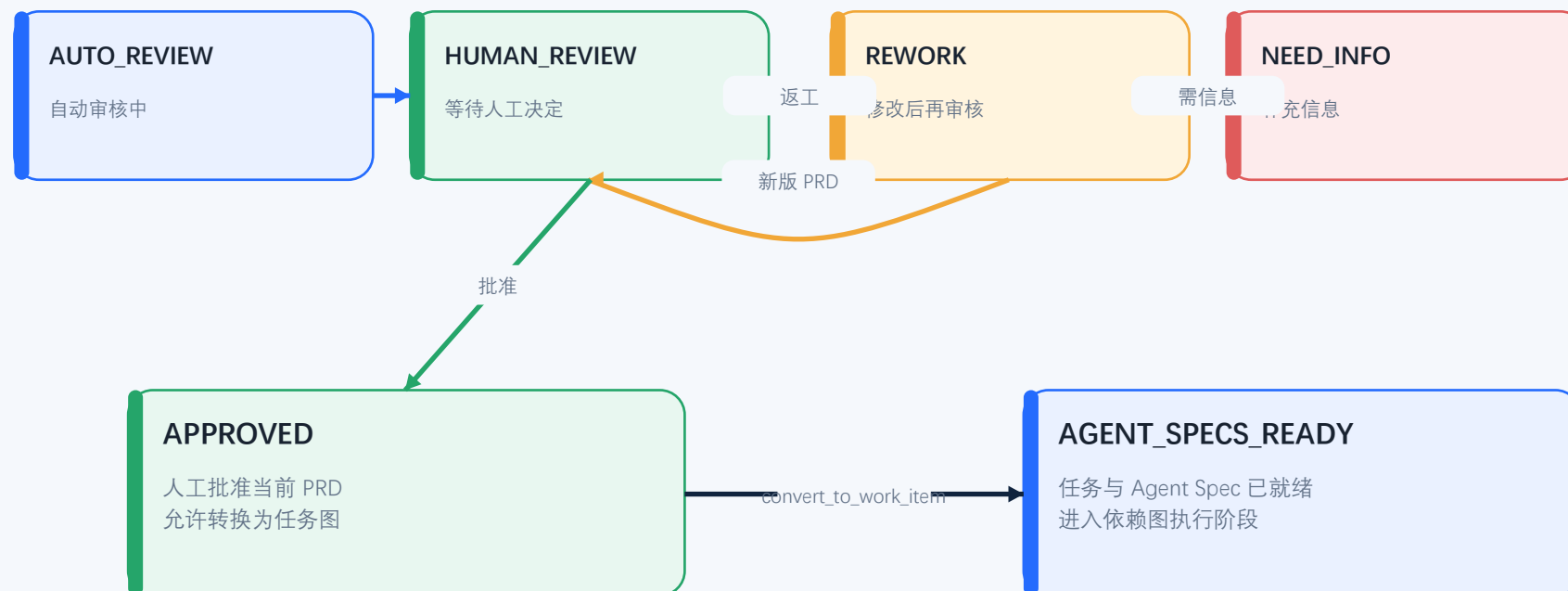
FSM 决定阶段，DAG 决定顺序，账本决定状态，两阶段命令保证每次推进可重试、可追踪、不会重复落库。

项目生命周期状态机

ProjectPhase 与 SpecStatus 组成复合状态；前端只显示后端允许的动作。

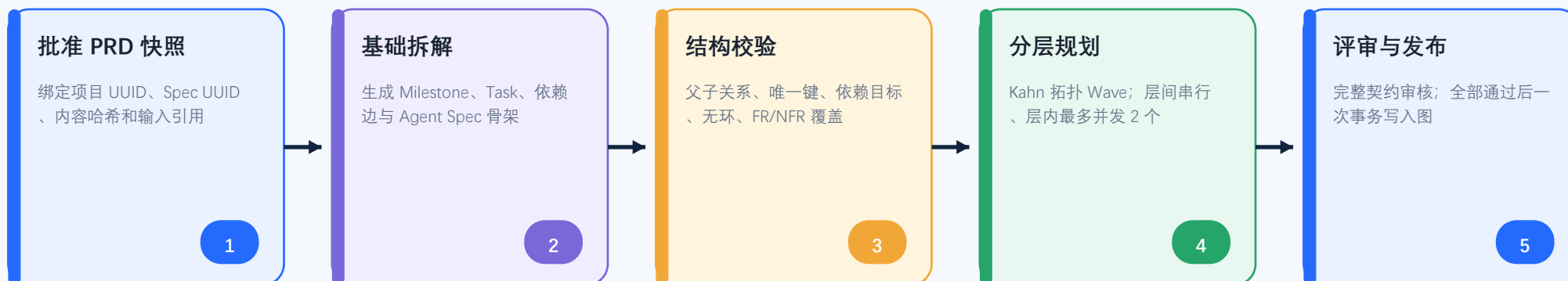


REVIEW 内部子状态



从批准的 PRD 编译出任务 DAG

先生成结构，再按拓扑层生成详细计划，最后统一审核和原子发布。



拓扑 Wave 示例



失败恢复规律

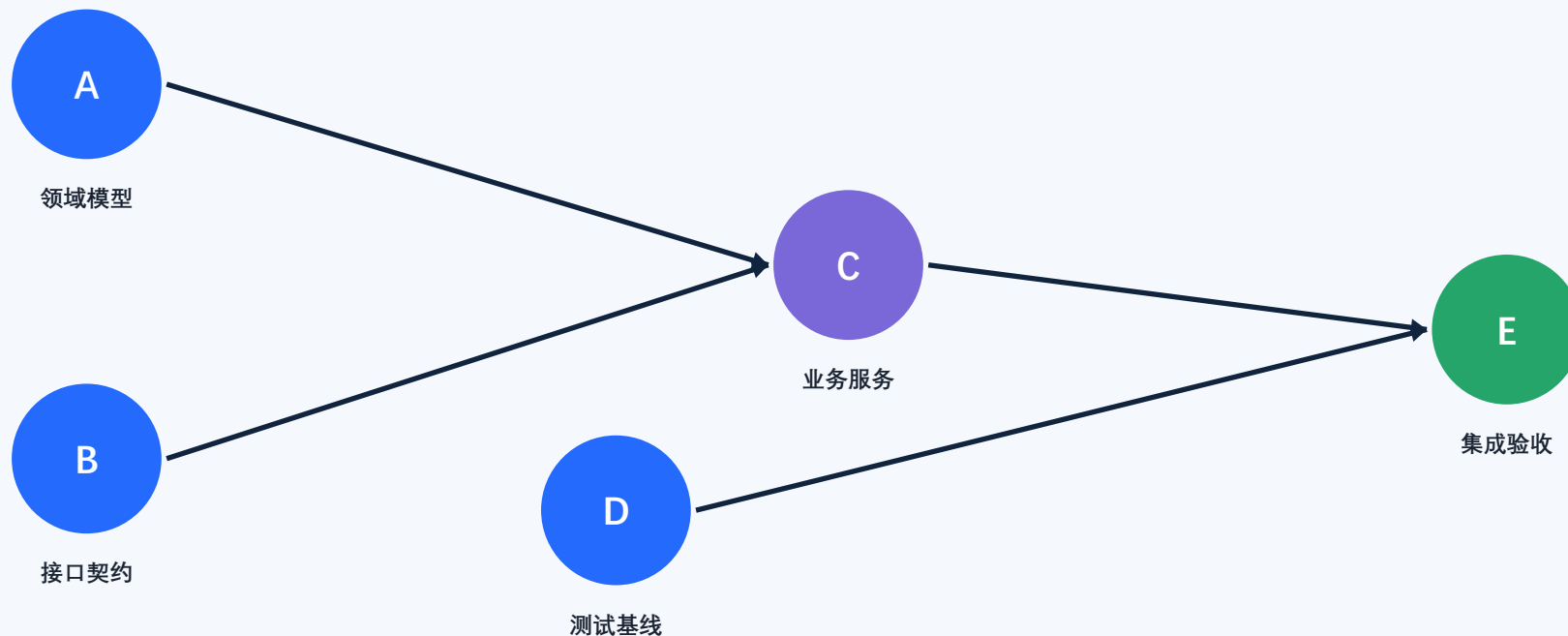
某个任务计划失败时，已成功的同级计划保留为 AgentCall 检查点；重试仅补齐缺失或失效的任务，不公开半成品 WorkItem。

依赖完成后逐步解锁任务

数据存“当前任务依赖谁”，显示时转换成“前置任务指向后继任务”。

启动条件

TASK + executable=true + status=todo + 所有前置任务均为 done



初始可启动集合

A、B、D 无未完成依赖；C 等待 A+B；E 等待 C+D

完成 A 和 B 后

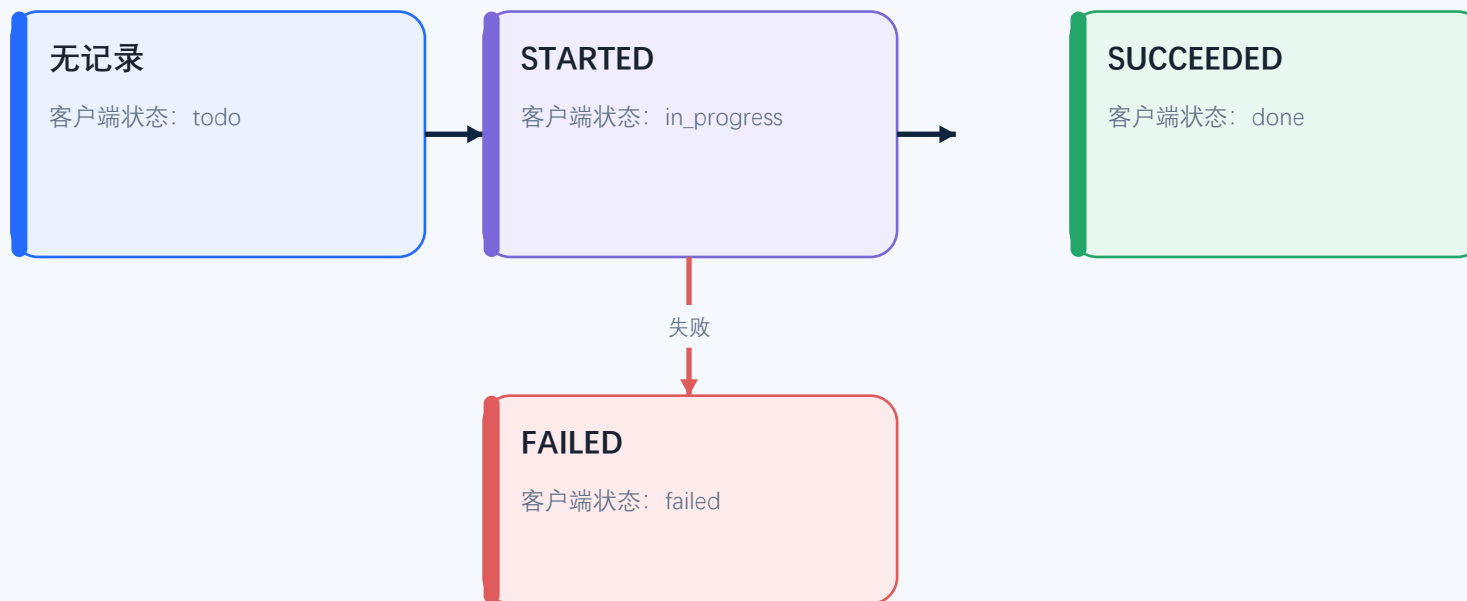
C 获得 start_task；状态来自最新 WorkItemRun

完成 C 和 D 后

E 被解锁，整条依赖链可以收敛

状态由追加式账本推导，而不是覆盖节点

每个操作新增一条 WorkItemRun；最新记录代表当前状态，历史完整保留。



一次任务命令的判定链



当前边界

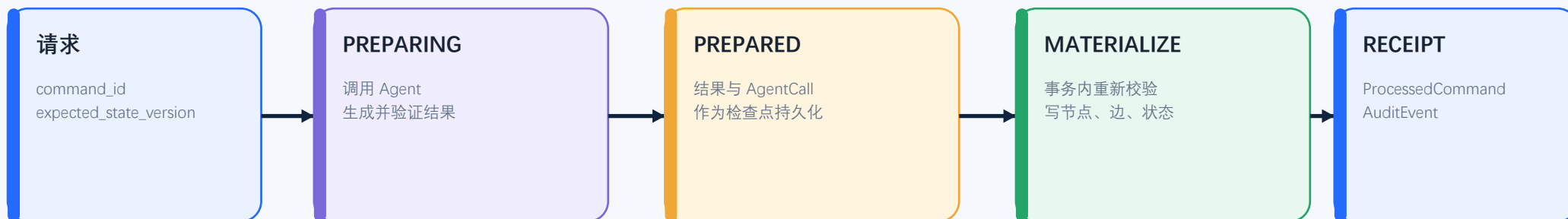
任务执行目前是依赖感知的人工推进：系统提供开始、完成、失败按钮，但不会自动拉起子 Agent、执行代码或自动完成任务。

当前规则漂移

投影层会给 failed / blocked 显示 start_task，但执行服务只允许 todo 启动。失败任务的“重新开始”目前会被后端拒绝。

让整套图可靠运转的命令引擎

外部工作与数据库提交分离，使每次推进都具备幂等、并发保护和审计证据。



幂等

相同 command_id + 相同输入
直接返回已保存结果

乐观锁

state_version 已变化时拒绝
旧页面请求

原子性

图节点、边、状态和审计在
一个事务中提交

恢复

超时后复用 PREPARED 与 A
gentCall 证据

最终运转公式

FSM 控制阶段 + DAG 控制顺序 + WorkItemRun 控制状态 + 两阶段命令控制一致性